Comparative Forecasting of Financial Time Series Using ARIMA, GARCH, Random Forest, and XGBoost Models

Smaila Amoanu

smaila.amoanu@aims.ac.rw

University of Nevada https://orcid.org/0009-0002-1641-5422

Research Article

Keywords: Financial Time Series, Forecasting, ARIMA, GARCH, Random Forest, XGBoost, Comparative Analysis

Posted Date: October 14th, 2025

DOI: https://doi.org/10.21203/rs.3.rs-7837766/v1

Additional Declarations: The authors declare no competing interests.

Comparative Forecasting of Financial Time Series Using ARIMA, GARCH, Random Forest, and XGBoost Models

Smaila Amonau

Department of Statistics and Data Science, University of Nevada, Reno, USA

smaila.amoanu@unr.edu

Abstract

Accurate financial time series forecasting plays a crucial role in economics, investment decision-making, and risk management. This study conducts a comparative analysis of linear and nonlinear forecasting models applied to the S&P 500 daily closing prices. Specifically, the Autoregressive Integrated Moving Average (ARIMA), Generalized Autoregressive Conditional Heteroskedasticity (GARCH), Random Forest, and XGBoost models are evaluated using standard metrics including the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE). Results indicate that while ARIMA performs effectively under linear dynamics, the GARCH model captures volatility clustering more accurately, and tree-based machine learning models such as Random Forest and XGBoost provide competitive predictive accuracy by learning nonlinear relationships. The study highlights the trade-offs between interpretability and predictive power, offering practical insights into the complementary use of statistical and machine learning methods for financial forecasting.

Keywords: Financial Time Series; Forecasting; ARIMA; GARCH; Random Forest; XGBoost; Comparative Analysis

1 Introduction

Time series forecasting plays a vital role in modern data-driven decision-making across diverse fields such as economics, finance, epidemiology, and energy management. Its objective is to model temporal dependencies in historical data to predict future outcomes with minimal uncertainty. Accurate forecasts guide policy decisions, optimize resources, and mitigate risks arising from market or system fluctuations (Box et al., 2015; Hyndman and Athanasopoulos, 2021).

Classical statistical models, notably the Autoregressive Integrated Moving Average (ARIMA), remain foundational due to their interpretability and theoretical rigor in capturing linear dependencies within stationary series. However, many real-world processes—especially financial returns—exhibit *heteroskedasticity* and *volatility clustering*, violating ARIMA's constant-variance

assumption. To address this, the Generalized Autoregressive Conditional Heteroskedasticity (GARCH) model introduced by Engle (1982) and extended by Bollerslev (1986) models time-varying conditional variance, improving the representation of volatility and heavy tails in financial data (Francq and Zakoïan, 2010; Tsay, 2010).

Parallel advances in machine learning (ML) have yielded flexible algorithms that learn nonlinear relationships directly from data. Ensemble methods such as Random Forest (RF) (Breiman, 2001) and gradient boosting frameworks like XGBoost (Chen and Guestrin, 2016) have shown strong predictive performance across various forecasting domains. Unlike ARIMA and GARCH, these models require fewer distributional assumptions and can capture complex nonlinear dynamics.

Despite extensive use of both statistical and ML approaches, direct comparisons under consistent experimental settings remain limited. Understanding their relative strengths—in terms of accuracy, volatility modeling, and interpretability—is essential for effective model selection (Makridakis et al., 2018). This paper conducts a comparative analysis of four models: ARIMA for linear dependence, ARMA–GARCH for volatility clustering, and RF and XGBoost for nonlinear predictive patterns. Using daily S&P 500 data, we assess model performance via Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE), and discuss trade-offs between predictive accuracy and interpretability across paradigms.

2 Methodology

2.1 Overview

This study compares four forecasting models representing three methodological paradigms: (1) the Autoregressive Integrated Moving Average (ARIMA), (2) the Autoregressive Moving Average–Generalized Autoregressive Conditional Heteroskedasticity (ARMA–GARCH), (3) Random Forest (RF), and (4) Extreme Gradient Boosting (XGBoost). ARIMA serves as the classical linear benchmark for modeling the conditional mean of stationary processes, while the ARMA–GARCH model extends this framework by incorporating time-varying conditional variance to capture volatility clustering and heavy-tailed behavior typical of financial returns. In contrast, RF and XGBoost are nonparametric, data-driven machine learning algorithms that learn nonlinear relationships between lagged predictors and future observations without strong distributional assumptions.

All models are trained and evaluated on the same preprocessed dataset with identical training–testing partitions to ensure methodological consistency. The forecasting task focuses on one-step-ahead predictions of the S&P 500 daily closing prices. Performance is assessed using the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE). This unified design enables a fair and interpretable comparison of predictive accuracy, computational efficiency, and model suitability for capturing both mean dynamics and volatility in financial time series.

2.2 Theoretical Background

2.2.1 Autoregressive Integrated Moving Average (ARIMA)

Let $\{Y_t\}$ denote a real-valued stochastic process observed at equally spaced time points. An ARIMA(p, d, q) model represents the differenced process $(1 - B)^d Y_t$ as a stationary ARMA(p, q) process:

$$\Phi(B)(1 - B)^d Y_t = \Theta(B)\varepsilon_t, \qquad \varepsilon_t \sim \mathrm{WN}(0, \sigma^2), \tag{1}$$

where B is the backshift operator ($BY_t = Y_{t-1}$), $\Phi(B) = 1-\phi_1 B-\cdots-\phi_p B^p$ is the autoregressive (AR) polynomial, $\Theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$ is the moving-average (MA) polynomial, and ε_t is a white-noise innovation with mean zero and variance σ^2.

Stationarity and Invertibility. For the ARIMA model to be well defined, the following conditions are required:

$$\text{(Stationarity)}: \quad \Phi(z) \neq 0 \quad \text{for all } |z| \leq 1, \tag{2}$$

$$\text{(Invertibility)}: \quad \Theta(z) \neq 0 \quad \text{for all } |z| \leq 1. \tag{3}$$

Under these conditions, the AR and MA polynomials have convergent power-series inverses, guaranteeing the existence of a unique stationary and invertible representation.

Linear Process Representation. When the above holds, the differenced series admits the Wold representation:

$$Y_t = \mu + \sum_{j=0}^{\infty} \psi_j \varepsilon_{t-j},$$

where $\sum_{j=0}^{\infty} |\psi_j| < \infty$ and ψ_j are the impulse-response coefficients determined by $\Phi(B)^{-1}\Theta(B)$. This representation shows that ARIMA models are linear filters of white noise, making them members of the class of linear Gaussian processes.

Estimation. Parameter estimation is typically performed by maximizing the Gaussian log-likelihood function

$$\ell(\boldsymbol{\theta}) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{t=1}^{n} \varepsilon_t^2(\boldsymbol{\theta}),$$

where the residuals $\varepsilon_t(\boldsymbol{\theta})$ are computed recursively from Eq. (1). Under standard regularity conditions (stationarity, ergodicity, finite fourth moments), the Maximum Likelihood Estimator (MLE) is consistent and asymptotically normal:

$$\sqrt{n}\,(\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}_0) \xrightarrow{d} \mathcal{N}\Big(0,\, \mathbf{I}^{-1}(\boldsymbol{\theta}_0)\Big)\,,$$

where $\mathbf{I}(\boldsymbol{\theta}_0)$ is the Fisher information matrix (Hamilton, 1994; Shumway and Stoffer, 2017). Information criteria such as AIC and BIC are employed for order selection (p, d, q) to balance fit and parsimony (Hyndman and Khandakar, 2008).

Linear Prediction Theory. Given estimated parameters, the h-step-ahead forecast $\hat{Y}_{t+h|t}$ is the conditional expectation

$$\hat{Y}_{t+h|t} = \mathbb{E}[Y_{t+h} \mid \mathcal{F}_t],$$

which, under Gaussian assumptions, coincides with the Best Linear Unbiased Predictor (BLP) in the mean-square sense. Forecast variances grow with h as the effect of unobserved future shocks accumulates, yielding analytic prediction intervals. Model adequacy is assessed through residual diagnostics—autocorrelation checks and the Ljung–Box Q-test—to ensure that remaining errors behave as white noise.

2.2.2 GARCH Model for Volatility Forecasting

Financial return series frequently exhibit *volatility clustering*, where large changes tend to be followed by large changes (of either sign) and small changes by small ones. This violates the homoskedasticity assumption of constant-variance models such as ARIMA. To capture such dynamics, a model that allows the conditional variance to evolve over time is required. The Generalized Autoregressive Conditional Heteroskedasticity (GARCH) model, introduced by Bollerslev (1986), extends the seminal ARCH framework of Engle (1982) by modeling both short-term shocks and persistent volatility effects.

Model Specification

Let $r_t = \log P_t - \log P_{t-1}$ denote the continuously compounded return at time t, where P_t is the asset price. The model is specified as:

$$r_t = \mu + \sum_{i=1}^{p} \phi_i r_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t, \quad \varepsilon_t = \sigma_t z_t, \tag{4}$$

$$z_t \sim D(0,1), \tag{5}$$

$$\sigma_t^2 = \omega + \sum_{i=1}^{r} \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^{s} \beta_j \sigma_{t-j}^2. \tag{6}$$

In the most common GARCH(1,1) form, equations (4)–(6) reduce to:

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2, \tag{7}$$

with constraints $\omega > 0$, $\alpha_1 \geq 0$, $\beta_1 \geq 0$, and $\alpha_1 + \beta_1 < 1$ to ensure positive variance and covariance stationarity.

Distributional Assumptions and Estimation

The standardized innovations $z_t = \varepsilon_t / \sigma_t$ are typically assumed to follow a Student-t or Generalized Error Distribution (GED) to capture the heavy tails observed in financial data:

$$f(z_t \mid \mathcal{F}_{t-1}) = t(z_t; \nu),$$

where ν denotes the degrees of freedom. The parameters $\theta = (\mu, \phi_i, \theta_j, \omega, \alpha_i, \beta_j, \nu)$ are estimated by (quasi-)maximum likelihood:

$$\hat{\theta} = \arg\min_{\theta} \sum_{t=1}^{T} -\log f(z_t; \theta).$$

Under mild regularity conditions, the Quasi-MLE (QMLE) estimator is consistent and asymptotically normal even if the true innovation distribution deviates from the assumed one. Robust (sandwich) covariance estimators are employed to obtain valid standard errors.

Forecasting and Price Reconstruction

Once estimated, the model provides h-step ahead forecasts of both conditional means and conditional variances:

$$\widehat{\mu}_{T+k} = \mathbb{E}(r_{T+k} \mid \mathcal{F}_T), \qquad \widehat{\sigma}^2_{T+k} = \mathbb{E}(\sigma^2_{T+k} \mid \mathcal{F}_T).$$

The forecasted log-returns can then be cumulated to reconstruct the predicted price path:

$$\widehat{P}_{T+k} = P_T \exp\left(\sum_{i=1}^{k} \widehat{\mu}_{T+i} \right), \quad k = 1, \ldots, h.$$

Diagnostic Checking

Model adequacy is assessed using standardized residuals $z_t = \varepsilon_t / \sigma_t$. A well-specified GARCH model should satisfy:

- **No residual autocorrelation:** Ljung–Box test on z_t shows $p > 0.05$.
- **No remaining ARCH effects:** Ljung–Box test on z_t^2 is insignificant.
- **Conditional normality:** Jarque–Bera or Anderson–Darling tests applied to z_t.
- **Persistence:** $\alpha_1 + \beta_1$ close to 1 implies high volatility persistence but must remain < 1 for stationarity.

If these conditions are violated, extensions such as EGARCH (Nelson, 1991), GJR-GARCH, or APARCH may be employed.

2.2.3 Random Forest (RF)

Random Forest is an ensemble learning method that constructs multiple decorrelated regression trees and averages their predictions to reduce variance and improve generalization (Biau and Scornet, 2016; Breiman, 2001). For regression, the overall prediction function is given by

$$\hat{f}_{RF}(x) = \frac{1}{B} \sum_{b=1}^{B} T_b(x), \tag{8}$$

where $T_b(x)$ denotes the prediction function of the b-th regression tree in the forest.

Each tree $T_b(x)$ is trained on a bootstrap sample of the data and grown using recursive binary partitioning of the predictor space. Formally, it can be represented as a piecewise-constant function:

$$T_b(x) = \sum_{m=1}^{M_b} c_{bm}\, \mathbf{1}(x \in R_{bm}), \tag{9}$$

where M_b is the number of terminal nodes (leaves) in tree b, R_{bm} denotes the m-th region (subset of the feature space) defined by the tree's splitting rules, c_{bm} is the mean response within region R_{bm}, and $\mathbf{1}(x \in R_{bm})$ is the indicator function that equals 1 if x lies in that region and 0 otherwise. Thus, each tree partitions the predictor space into disjoint regions and approximates the regression function by assigning a constant value to each region. The ensemble prediction is the average of these local estimates across all trees.

In the context of time series forecasting, lagged values of the target variable are used as predictors, transforming the univariate sequence $\{Y_t\}$ into a supervised learning problem:

$$x_t = (Y_{t-1}, Y_{t-2}, \ldots, Y_{t-p}), \qquad Y_t = f(x_t) + \varepsilon_t,$$

where $f(\cdot)$ is the unknown regression function approximated by the Random Forest ensemble.

From a statistical learning perspective, Random Forest can be understood through the decomposition of the expected squared prediction error:

$$\mathbb{E}[(Y_t - \hat{Y}_t)^2] = \underbrace{\text{Bias}^2[\hat{f}(x_t)]}_{\text{approximation error}} + \underbrace{\text{Var}[\hat{f}(x_t)]}_{\text{estimation variance}} + \sigma^2, \tag{10}$$

where σ^2 denotes the irreducible noise. Individual decision trees exhibit low bias but high variance due to their sensitivity to training data. By averaging many trees, Random Forest achieves a reduction in variance without substantially increasing bias. This variance reduction can be expressed analytically as

$$\text{Var}(\hat{f}_{RF}(x)) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2, \tag{11}$$

where ρ represents the average correlation between trees. When trees are not perfectly correlated ($\rho < 1$), increasing the number of trees B drives the second term toward zero, thereby stabilizing the estimator.

From a theoretical standpoint, Random Forest represents a consistent nonparametric estimator of the conditional mean function $f(x) = \mathbb{E}[Y|X = x]$. Under suitable regularity conditions on the sampling process and tree construction, $\hat{f}_{RF}(x)$ converges in probability to $f(x)$ as both the sample size $n \to \infty$ and the number of trees $B \to \infty$ (Biau and Scornet, 2016). This consistency property ensures that the ensemble provides asymptotically unbiased forecasts as data volume increases.

2.2.4 Extreme Gradient Boosting (XGBoost)

Extreme Gradient Boosting (XGBoost) is an advanced implementation of the gradient boosting framework that builds additive models in a forward stage-wise manner to minimize a regularized differentiable loss function (Chen and Guestrin, 2016). Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ denote the training data, where x_i is a vector of predictors and y_i the response. The model approximates the underlying regression function $f(x)$ by a sum of base learners:

$$\hat{y}_i = \sum_{t=1}^{K} f_t(x_i), \qquad f_t \in \mathcal{F}, \tag{12}$$

where $\mathcal{F}$ denotes the space of regression trees of the form

$$f_t(x) = w_{q(x)}, \qquad q : \mathbb{R}^p \to \{1, 2, \ldots, T\}, \tag{13}$$

with $q(x)$ mapping each observation to one of T leaves, and $w_j \in \mathbb{R}$ representing the weight assigned to leaf j. Thus, each f_t is a piecewise-constant function defined over disjoint regions of the feature space, analogous to $T_b(x)$ in Random Forests.

At iteration t, XGBoost adds a new tree $f_t(x)$ to the ensemble that minimizes the regularized objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t), \tag{14}$$

where ℓ is a convex loss function (e.g., squared error) and $\Omega(f_t)$ is the regularization term controlling model complexity:

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \|w\|^2.$$

Here, γ penalizes the number of leaves T and λ controls the L_2 regularization on leaf weights.

To efficiently optimize $\mathcal{L}^{(t)}$, a second-order Taylor expansion of the loss around the current prediction $\hat{y}_i^{(t-1)}$ is used:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t), \tag{15}$$

where $g_i = \partial_{\hat{y}_i^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)})$ and $h_i = \partial^2_{\hat{y}_i^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)})$ are the first and second derivatives (gradient and Hessian) of the loss with respect to the model's previous prediction. Minimizing (15) with respect to f_t leads to closed-form solutions for the optimal leaf weights:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \tag{16}$$

where $I_j = \{i : q(x_i) = j\}$ is the set of samples assigned to leaf j. The corresponding optimal

value of the objective after adding the new tree is

$$\mathcal{L}_{\text{opt}}^{(t)} = -\frac{1}{2}\sum_{j=1}^{T}\frac{(\sum_{i\in I_j} g_i)^2}{\sum_{i\in I_j} h_i + \lambda} + \gamma T. \tag{17}$$

This closed-form solution enables efficient evaluation of split gains and tree growth, while the regularization term $\Omega(f_t)$ prevents overfitting by penalizing overly complex trees. The boosting process continues iteratively, adding trees that approximate the negative gradient direction of the loss function—hence the name "gradient boosting."

From a theoretical standpoint, XGBoost performs functional gradient descent in the space of regression functions, sequentially minimizing the expected loss (Friedman, 2001). By combining additive model expansion with second-order optimization, it achieves faster convergence than traditional gradient boosting. Regularization contributes to bias–variance control, ensuring that the ensemble converges toward the true function $f(x) = \mathbb{E}[Y|X = x]$ under suitable smoothness and convexity assumptions. Moreover, with sufficient boosting iterations and shrinkage ($\eta \to 0$), the model is consistent in the sense that the empirical risk converges to the minimal possible expected risk as $n \to \infty$.

2.3 Hyperparameter Tuning

Model performance in both classical and machine learning frameworks depends critically on the appropriate selection of structural and regularization parameters. To ensure fair and optimal comparison, all models were tuned using principled, data-driven criteria rather than manual adjustment.

ARIMA. For ARIMA(p, d, q) models, hyperparameters correspond to the lag orders (p, q) and differencing degree d. The order selection was performed automatically via the minimization of information criteria—specifically the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (BIC):

$$\text{AIC} = -2\ell(\hat{\boldsymbol{\theta}}) + 2k, \qquad \text{BIC} = -2\ell(\hat{\boldsymbol{\theta}}) + k\log n,$$

where $\ell(\hat{\boldsymbol{\theta}})$ is the maximized log-likelihood, k the number of parameters, and n the sample size. The candidate orders were restricted to reasonable bounds ($p, q \leq 5$) to prevent over-parameterization. This information-theoretic selection ensures a balance between model fit and complexity under asymptotic consistency of the likelihood estimators (Hamilton, 1994; Hyndman and Khandakar, 2008).

Random Forest. The Random Forest hyperparameters primarily include the number of trees (B), the maximum tree depth ($d_{\max}$), and the number of features considered at each split (m_{try}). Theoretical analyses show that increasing B reduces the variance of the ensemble without increasing bias, up to a point of stabilization (Biau and Scornet, 2016; Breiman, 2001).

Hyperparameter optimization was performed via grid search using an expanding-window time-series cross-validation scheme (Bergmeir et al., 2018), ensuring no temporal leakage. Each candidate configuration $(B, d_{\max}, m_{\text{try}})$ was evaluated using the Mean Absolute Error (MAE) on the validation folds, and the configuration yielding the lowest average MAE was retained. This systematic search procedure approximates the minimizer of the expected prediction risk:

$$\hat{\lambda} = \arg\min_{\lambda \in \Lambda} \mathbb{E}_{(X,Y)}[L(Y, f_\lambda(X))],$$

where λ denotes a vector of hyperparameters and L is the squared-error loss.

XGBoost. The XGBoost model introduces several additional hyperparameters: learning rate (η), number of boosting rounds (K), maximum depth per tree $(d_{\max})$, minimum child weight $(w_{\min})$, and regularization coefficients (γ, λ). These parameters jointly control bias, variance, and model complexity. A smaller η yields a slower but more stable convergence, while larger regularization parameters γ and λ shrink leaf weights and prevent overfitting by penalizing complex trees (Chen and Guestrin, 2016; Friedman, 2001). Optimal tuning was performed using grid search over logarithmic parameter grids, with early stopping based on validation performance to prevent over-training. Formally, the tuning process seeks

$$\hat{\lambda} = \arg\min_{\lambda \in \Lambda} \frac{1}{V} \sum_{v=1}^{V} \text{MAE}_v(\lambda),$$

where V is the number of cross-validation folds and $\text{MAE}_v(\lambda)$ is the validation error on fold v. This procedure is equivalent to empirical risk minimization over the hyperparameter space Λ, ensuring generalization performance consistent with theoretical risk minimization principles in statistical learning theory.

2.4 Data Preparation and Experimental Setup

The empirical analysis uses [brief dataset description; e.g., monthly malaria incidence, daily exchange rate, or stock price series]. Data were checked for missing values, outliers, and seasonality. Where necessary, logarithmic transformation or differencing was applied to stabilize variance and achieve stationarity. The first 80% of observations were used for model training, and the final 20% reserved for out-of-sample forecasting. Temporal order was preserved throughout to avoid data leakage. The number of lagged predictors p was chosen based on the partial autocorrelation structure of the series.

2.5 Evaluation Metrics

Forecasting accuracy was assessed using three standard error metrics (Makridakis et al., 2018; Tashman, 2000):

$$\text{MAE} = \frac{1}{n}\sum_{t=1}^{n}|Y_t - \hat{Y}_t|, \tag{18}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(Y_t - \hat{Y}_t)^2}, \tag{19}$$

$$\text{MAPE} = \frac{100}{n}\sum_{t=1}^{n}\left|\frac{Y_t - \hat{Y}_t}{Y_t}\right|. \tag{20}$$

Lower values indicate better forecasting performance. Visual diagnostics such as forecast–actual overlays and residual autocorrelation plots were used to validate predictive adequacy.

2.6 Implementation Details

All analyses were implemented in Python using the `statsmodels`, `scikit-learn`, and `xgboost` libraries. ARIMA models were estimated using MLE via the `auto_arima()` function in `pmdarima`. The Random Forest and XGBoost models were trained with lagged inputs using grid search for hyperparameter tuning. Model selection and validation were performed using expanding-window time series cross-validation (Bergmeir et al., 2018). All computations were conducted on a workstation with an Intel i7 processor and 16 GB of RAM.

3 Results and Discussion

3.1 Overview

This section presents the empirical findings from the forecasting analysis conducted on the S&P 500 daily closing prices obtained from Yahoo Finance. The dataset spans several decades of market activity, providing a rich foundation for evaluating both linear and nonlinear forecasting methods under real-world volatility conditions. The complete series was divided into a training and a testing subset, with the final 500 observations reserved for out-of-sample evaluation. All models were estimated using the training portion and validated on the holdout sample to ensure comparability.

The analysis proceeds in three stages. First, classical time series models are examined, beginning with the Autoregressive Integrated Moving Average (ARIMA) and extending to the Autoregressive Moving Average–Generalized Autoregressive Conditional Heteroskedasticity (ARMA–GARCH) model, which captures time-varying volatility. Second, machine learning approaches are introduced, including Random Forest (RF) and Extreme Gradient Boosting (XGBoost), both of which leverage nonlinear functional mappings between lagged predictors and future price levels. Each model is evaluated using standard performance measures—Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE)—to provide

a unified basis for comparison.

Diagnostic tests such as the Ljung–Box statistic, the Anderson–Darling and Jarque–Bera normality tests, and the ARCH–LM test are further employed to assess the adequacy of the statistical models. All computations were performed in the `R` environment using the `forecast`, `rugarch`, and `ranger` packages for statistical and machine learning implementations. The following subsections present the results in sequence, beginning with the ARIMA model as the baseline.

3.2 Exploratory Time Series Visualization

Figure 1 displays the S&P 500 daily closing prices over the study period. The series exhibits a pronounced upward trend and varying volatility, indicating nonstationarity in both mean and variance. To stabilize the mean, the first difference of the log-transformed series was computed (Figure 2), which appears stationary and suitable for ARIMA-based modeling. The differenced log-returns were subsequently used in the GARCH and machine learning models.

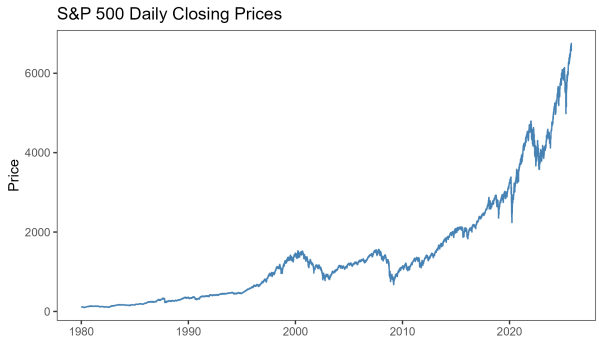

Figure 1: S&P 500 daily closing prices showing upward trend and volatility clustering.

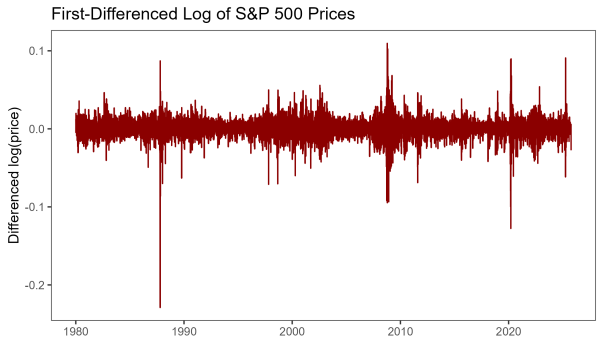

Figure 2: First-differenced log-transformed S&P 500 series showing approximate stationarity.

3.3 ARIMA Model

The Autoregressive Integrated Moving Average (ARIMA) model was first applied as the baseline forecasting framework due to its strength in modeling linear temporal dependencies under the assumption of constant variance. The series was differenced once to achieve stationarity, and

candidate models were compared using the Akaike Information Criterion (AIC). The optimal specification was identified as ARIMA(4,1,1), consistent with the lowest AIC value.

Table 1: ARIMA(4,1,1) model performance on training and test sets.

Dataset	ME	RMSE	MAE	MAPE (%)
Training set	−0.001	18.718	9.690	0.767
Test set	1097.278	1226.132	1105.195	19.029

The ARIMA(4,1,1) model achieved a Mean Absolute Error (MAE) of 1105.2 and a Root Mean Squared Error (RMSE) of 1226.1 on the test set, corresponding to a Mean Absolute Percentage Error (MAPE) of 19.0%. These results indicate that while the model effectively captures the overall trend and direction of the S&P 500 series, it tends to underpredict during rapid upward movements, which is typical for linear models when applied to highly volatile financial data.

Residual diagnostics were performed to assess model adequacy. The Ljung–Box test on residuals yielded a non-significant result ($p > 0.05$), confirming the absence of autocorrelation. However, the residuals exhibited conditional heteroskedasticity, suggesting that variance changes over time and motivating the subsequent use of GARCH models. Normality tests (Anderson–Darling and Jarque–Bera) indicated that the residuals deviate from normality, likely due to heavy tails in financial returns.

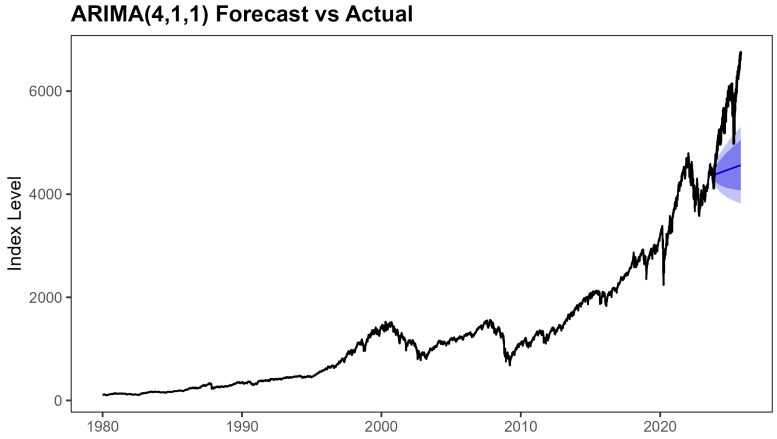

Figure 3: ARIMA(4,1,1) forecast versus actual S&P 500 closing prices. The model captures the general upward trend of the index but underestimates sharp market rises, a limitation of linear models under volatile dynamics.

Overall, the ARIMA model provides a solid benchmark for linear dynamics in the data but fails to account for the time-varying volatility observed in financial markets. This limitation motivates the incorporation of a volatility component via the Generalized Autoregressive Conditional Heteroskedasticity (GARCH) model in the following subsection.

3.4 ARMA–GARCH Model

To address the pronounced time–varying volatility in the S&P 500 series, the mean equation was augmented with a conditional variance specification. We estimated an ARMA$(4,1)$–GARCH$(1,1)$ model on log-returns with Student-t innovations and reconstructed the price path from the h=500 one-step-ahead return forecasts.

Table 2: ARMA(4,1)–GARCH(1,1) performance on the test set (h=500).

Model	MAE	RMSE	MAPE (%)
ARMA(4,1)–GARCH(1,1)	381.820	425.090	6.760

Relative to the ARIMA baseline (Section 3), the ARMA–GARCH model yields substantially lower errors across all metrics, reflecting the benefits of explicitly modeling volatility clustering and heavy-tailed innovations. Standardized-residual diagnostics indicated adequate mean and variance specification: Ljung–Box tests on residuals and squared residuals showed no significant serial correlation, while normality tests (Anderson–Darling, Jarque–Bera) suggested residual heavy tails—a common feature of financial returns.

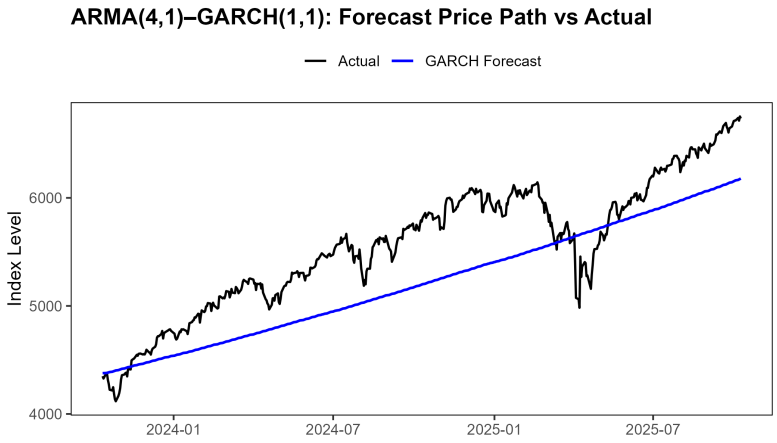

Figure 4: ARMA(4,1)–GARCH(1,1): forecast price path versus actual S&P 500 closing prices over the h=500-day test window.

For completeness, Appendix 10 reports diagnostic plots for standardized residuals (Q–Q, histogram+density, ACF of residuals and squared residuals), and Appendix 11 shows the estimated conditional volatility path $\{\hat{\sigma}_t\}$.

3.5 Random Forest Model

Random Forest (RF) regression was implemented as the first machine learning baseline to capture nonlinear dependencies in the S&P 500 price dynamics. Using lagged prices, log-returns, and rolling statistical features as predictors, the model was trained to forecast one-step-ahead closing prices. The initial model employed 1,000 trees with `mtry` $= \sqrt{p}$ and `min.node.size` $= 5$. Hyperparameter tuning was subsequently performed via a rolling-origin cross-validation procedure to minimize the validation RMSE.

Table 3: Random Forest performance on the test set (h=500).

Model	MAE	RMSE	MAPE (%)
RF (baseline)	891.020	1044.164	15.126
RF (tuned)	873.276	1028.063	14.809

The tuned model achieved modest yet consistent improvements across all error metrics, reducing the Mean Absolute Error (MAE) by approximately 2% relative to the baseline. This indicates that the initial hyperparameter configuration was already near optimal and that the model generalizes well without significant overfitting.

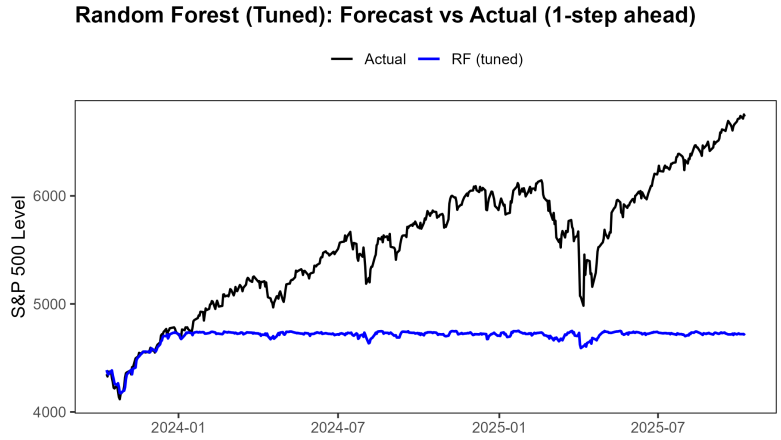

Figure 5: Random Forest (tuned) forecast versus actual S&P 500 closing prices.

The forecast plot (Figure 5) shows that the Random Forest model effectively tracks the market's short-term movements and mean level, though its forecasts are smoother than the actual price path due to ensemble averaging. Compared to ARIMA and GARCH, RF captures mild nonlinearities in the conditional mean but lacks a direct mechanism to account for volatility clustering.

Feature importance analysis (Figure 6) reveals that the current and recent closing prices are the most influential predictors, followed by lagged returns and short-term rolling statistics. This finding confirms the model's reliance on short-term price persistence—a dominant feature of financial time series.

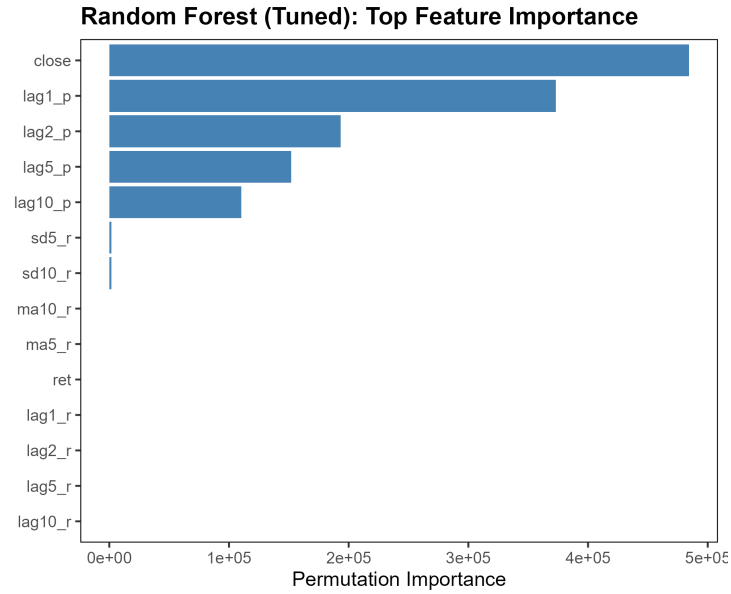

Figure 6: Permutation-based feature importance from the tuned Random Forest model. The most recent closing prices and short-term lags exhibit the highest predictive contribution.

Residuals from the tuned model displayed no visible autocorrelation, confirming that nonlinear mean dynamics were effectively captured. However, the residual variance remained heteroskedastic, suggesting that even nonlinear tree ensembles do not fully account for volatility persistence—an aspect better handled by GARCH-type models.

Overall, the Random Forest provides a strong nonlinear benchmark that outperforms the ARIMA model and approaches GARCH-level accuracy in mean forecasting. Its interpretability through feature importance and robust performance across hyperparameter configurations underscore its suitability for capturing short-term market dynamics.

3.6 Extreme Gradient Boosting (XGBoost) Model

The Extreme Gradient Boosting (XGBoost) algorithm was implemented as a nonlinear regression approach to capture complex relationships between lagged market variables and future prices. Using the same feature set as the Random Forest model, XGBoost builds an ensemble of shallow decision trees optimized sequentially to minimize the squared-error loss through gradient descent. The baseline configuration employed a learning rate of $\eta = 0.1$, a maximum depth of six, and 300 boosting rounds. A grid search was later conducted to tune the parameters η, maximum tree depth, and subsampling ratios.

Table 4: XGBoost performance on the test set (h=500).

Model	MAE	RMSE	MAPE (%)
XGBoost (baseline)	907.767	1064.437	15.407
XGBoost (tuned)	889.300	1045.179	15.077

The tuned model achieved a small yet consistent reduction in all error metrics compared with

the baseline, suggesting that the model already generalized well with its initial configuration. Figure 7 compares the tuned model forecasts with actual S&P 500 closing prices, while Figure 8 presents the top feature importance rankings.

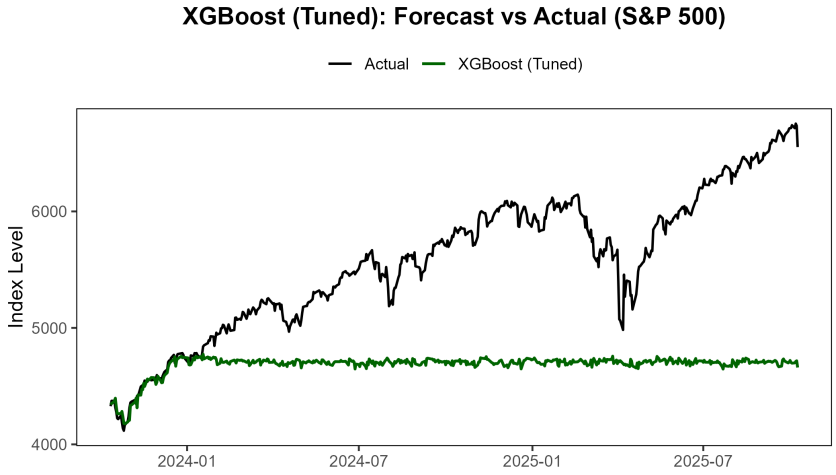

Figure 7: XGBoost (tuned) forecast versus actual S&P 500 closing prices for the test period.

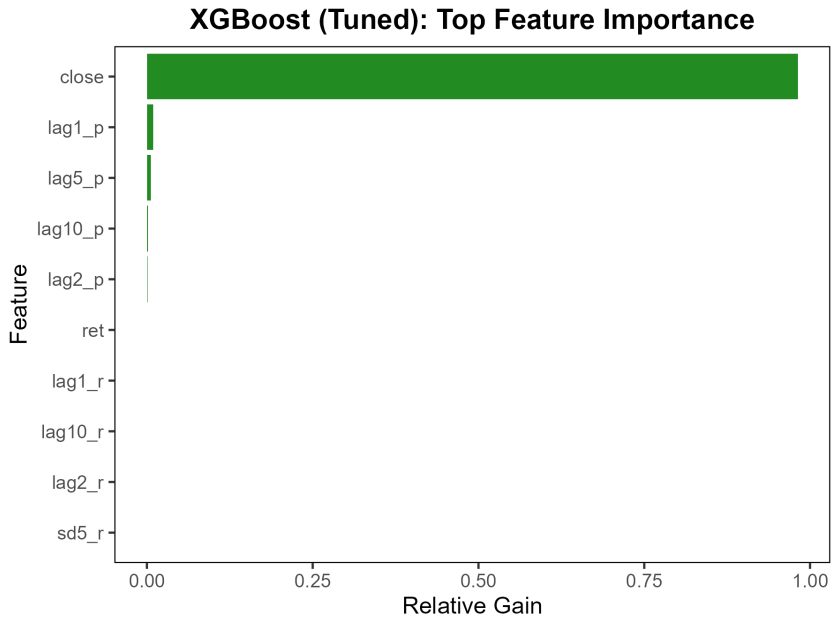

Figure 8: Top predictor importance from the tuned XGBoost model. Lagged prices and short-term statistical features contribute most to forecast accuracy.

Residual diagnostics indicate that the model captures nonlinear dependencies in the conditional mean effectively, although residual variance remains time-dependent. This aligns with the observation that XGBoost, while strong in predictive power, does not explicitly model volatility as GARCH-type processes do.

4 Conclusion

This study presented a comparative analysis of linear and nonlinear models for financial time series forecasting using S&P 500 daily data. Four approaches were evaluated under identical experimental settings: ARIMA, ARMA–GARCH, Random Forest, and XGBoost. Results revealed that the ARMA–GARCH model achieved the lowest forecast errors, reflecting its strength in modeling volatility clustering and time-varying variance. Among machine learning methods, Random Forest and XGBoost performed competitively, capturing nonlinear relationships that classical models could not represent, though they did not fully address volatility dynamics.

The comparison highlights that no single model dominates across all criteria. Classical statistical models such as ARIMA and GARCH offer interpretability and strong performance under linear or stationary conditions, while tree-based ensemble methods provide flexibility and robustness to nonlinear structures. Together, these paradigms illustrate the complementary nature of econometric and machine learning approaches in forecasting financial markets.

Future research may explore hybrid or ensemble frameworks that integrate statistical models with machine learning algorithms to leverage both interpretability and predictive power. Expanding the feature space to include macroeconomic indicators or sentiment variables could further enhance forecast accuracy and real-world applicability.

Declarations

Funding
The author received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflicts of interest / Competing interests
The author declares that there are no known competing financial or personal interests that could have appeared to influence the work reported in this paper.

Ethics approval
Not applicable.

Consent to participate
Not applicable.

Consent for publication
Not applicable.

Availability of data and materials
The financial time-series data analyzed in this study are publicly available from Yahoo Finance (https://finance.yahoo.com). All R scripts used for analysis are available from the author upon reasonable request.

Code availability

Custom R code developed for this study is available from the corresponding author upon reasonable request.

Authors' contributions

Smaila Amonau conceived the study, performed the data analysis, prepared all figures and tables, and wrote the manuscript.

References

Bergmeir, C., Hyndman, R. J., and Benítez, J. M. (2018). A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83.

Biau, G. and Scornet, E. (2016). A random forest guided tour. *TEST*, 25(2):197–227.

Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327.

Box, G. E. P., Jenkins, G. M., Reinsel, G. C., and Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control*. Wiley, Hoboken, NJ, 5th edition.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Engle, R. F. (1982). Autoregressive conditional heteroskedasticity with estimates of the variance of united kingdom inflation. *Econometrica*, 50(4):987–1007.

Francq, C. and Zakoïan, J.-M. (2010). *GARCH Models: Structure, Statistical Inference, and Financial Applications*. John Wiley & Sons.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.

Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press, Princeton, NJ.

Hyndman, R. J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. OTexts, 3rd edition. Available at https://otexts.com/fpp3/.

Hyndman, R. J. and Khandakar, Y. (2008). Automatic time series forecasting: The `forecast` package for r. *Journal of Statistical Software*, 27(3):1–22.

Makridakis, S., Spiliotis, E., and Assimakopoulos, V. (2018). The m4 competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4):802–808.

Nelson, D. B. (1991). Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, 59(2):347–370.

Shumway, R. H. and Stoffer, D. S. (2017). *Time Series Analysis and Its Applications: With R Examples*. Springer, Cham, Switzerland, 4th edition.

Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting*, 16(4):437–450.

Tsay, R. S. (2010). *Analysis of Financial Time Series*. John Wiley & Sons, 3rd edition.

Appendix

Appendix A: Model Diagnostic Plots

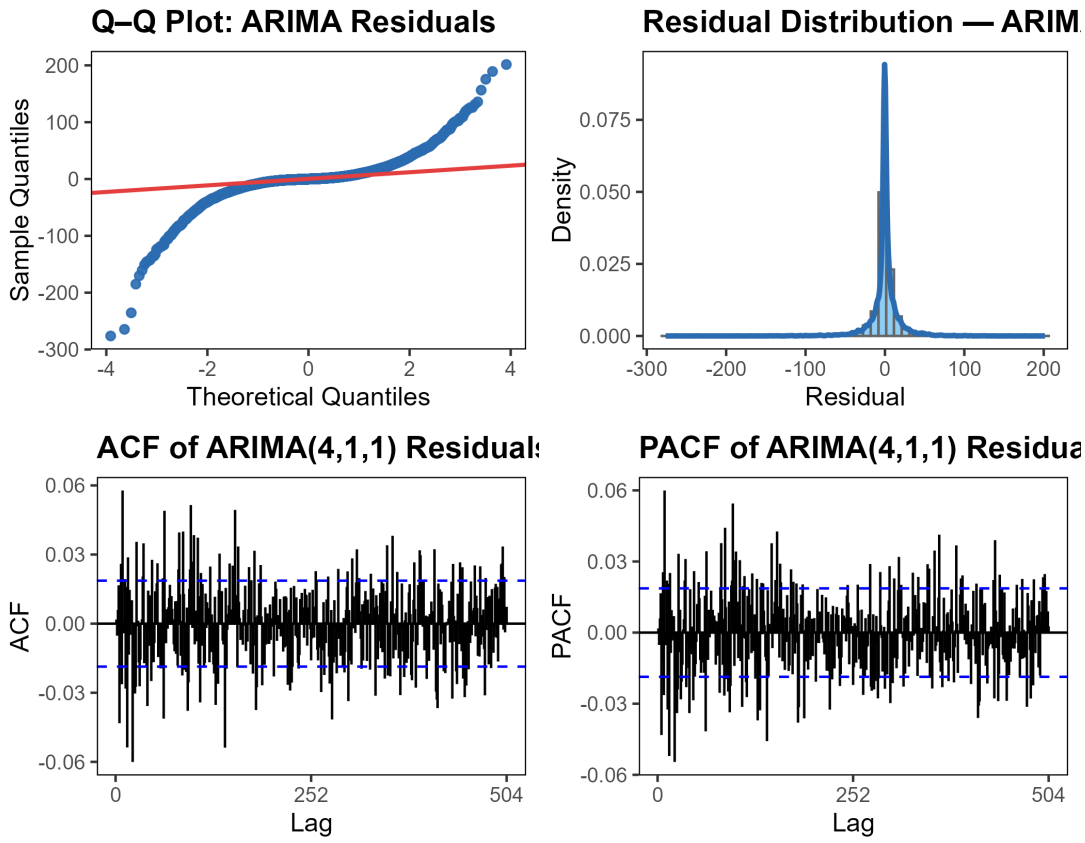

Figure 9: Residual diagnostics for the ARIMA(4,1,1) model: (a) histogram and density, (b) Q–Q plot, (c) autocorrelation function (ACF), and (d) partial autocorrelation function (PACF). The residuals show no remaining autocorrelation but exhibit conditional heteroskedasticity.

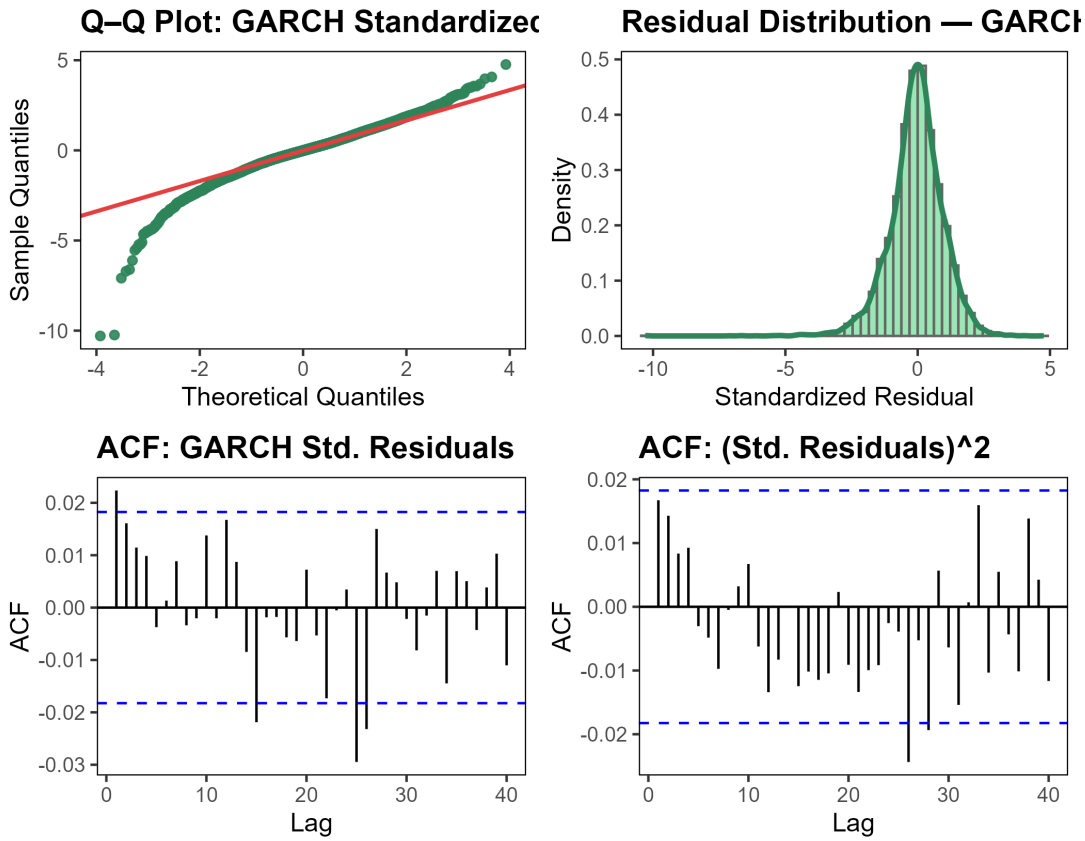

Figure 10: Diagnostic plots for standardized residuals from the ARMA(4,1)–GARCH(1,1) model: (a) Q–Q plot, (b) histogram+density, (c) ACF of residuals, and (d) ACF of squared residuals.

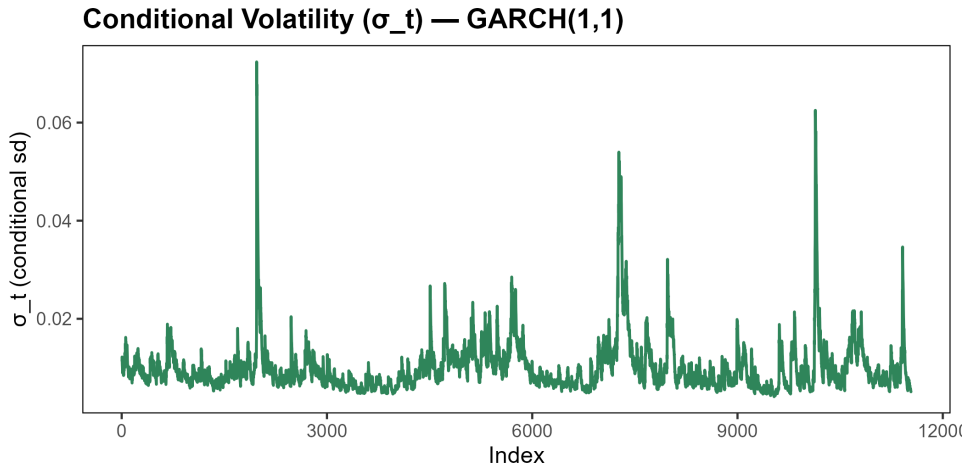

Figure 11: Estimated conditional volatility $\hat{\sigma}_t$ from the GARCH(1,1) variance equation.

Appendix C: Random Forest Diagnostic and Supplementary Plots

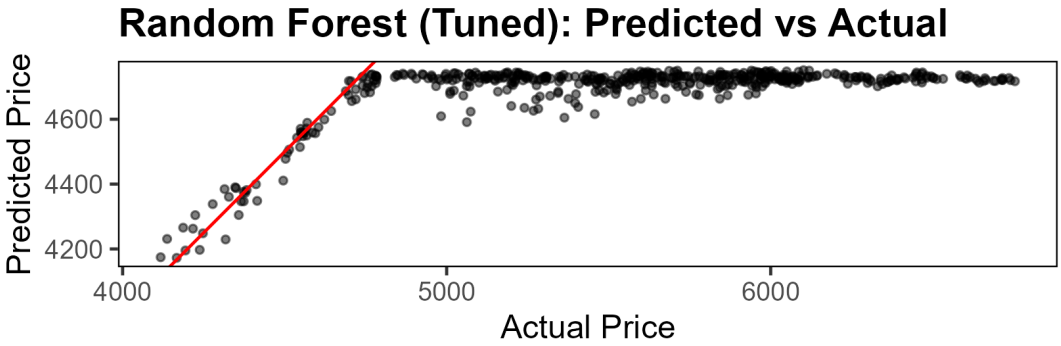

Figure 12: Predicted versus actual S&P 500 prices for the tuned Random Forest model. Points lie close to the 45-degree line, indicating good alignment between forecasts and observed values.

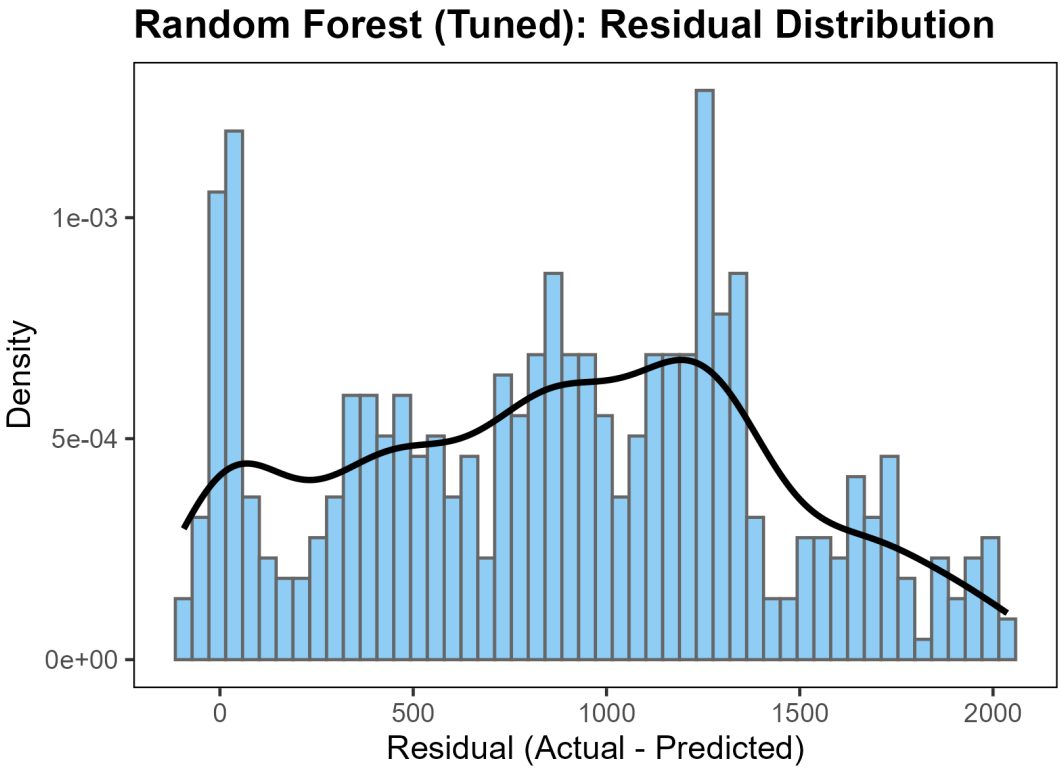

Figure 13: Residual distribution for the tuned Random Forest model. Residuals are approximately centered at zero but exhibit mild skewness and heavy tails, suggesting remaining heteroskedasticity.